

Math Foundations: Calculus and Optimization

Focus area A (flagged weak area)

This is the math under SL3 (backpropagation), OL1–OL2 (randomized and gradient-based optimization), and SL6 (the SVM constrained optimization). Backprop is the chain rule; AdamW is gradient descent with patches; the SVM is a Lagrangian. Close these gaps in Week 0. Log what stalls you in `math_gaps_log.md`.

1 Derivatives: Scalar Refresher

For $f : \mathbb{R} \rightarrow \mathbb{R}$, the **derivative** at x is the limit

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Geometrically, this is the slope of the tangent. Critical identities you will use:

$$\begin{aligned} (c)' &= 0, \\ (x^n)' &= nx^{n-1}, \\ (e^x)' &= e^x, \\ (\ln x)' &= 1/x, & x > 0, \\ (\sin x)' &= \cos x, & (\cos x)' = -\sin x, \\ (fg)' &= f'g + fg', & \text{product rule,} \\ (f/g)' &= (f'g - fg')/g^2, & \text{quotient rule,} \\ (f(g(x)))' &= f'(g(x))g'(x), & \text{chain rule.} \end{aligned}$$

The **second derivative** $f''(x)$ tells you curvature: $f'' > 0$ means convex, cup-shaped locally; $f'' < 0$ means concave; $f'' = 0$ is the inflection point between them.

2 Gradients: The Multivariate Generalization

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **gradient** is the column vector of partial derivatives:

$$\nabla f(x) = \begin{bmatrix} \partial f / \partial x_1 & \partial f / \partial x_2 & \cdots & \partial f / \partial x_n \end{bmatrix}^\top.$$

Two facts drive everything in this course:

- The gradient **points in the direction of steepest ascent**. $-\nabla f$ is therefore the direction of steepest descent, which is the premise of gradient descent.
- At a **minimum, maximum, or saddle point**, $\nabla f = 0$. Solving $\nabla f(x) = 0$ is how you derive closed-form estimators, including the OLS normal equations and the MLE computations in the probability notes.

Why steepest ascent? The directional derivative of f along unit vector u is

$$D_u f(x) = \nabla f(x) \cdot u = \|\nabla f(x)\| \|u\| \cos \theta = \|\nabla f\| \cos \theta.$$

This is maximized when $\cos \theta = 1$, i.e. u is aligned with ∇f . So ∇f is the direction of steepest ascent, and its magnitude is the rate of that ascent. Cauchy–Schwarz is doing the work here.

2.1 Jacobians and Hessians

For $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, vector-valued, the **Jacobian** generalizes the gradient:

$$(J_f)_{ij} = \frac{\partial f_i}{\partial x_j}, \quad J_f \in \mathbb{R}^{m \times n}.$$

The gradient is the Jacobian’s transpose when $m = 1$.

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **Hessian** is the matrix of second partials:

$$(H_f)_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}, \quad H_f \in \mathbb{R}^{n \times n},$$

which is symmetric by Clairaut’s theorem when the relevant derivatives are continuous.

The Hessian classifies critical points where $\nabla f = 0$:

Hessian property	Critical point type
PD, all $\lambda_i > 0$	Local minimum
ND, all $\lambda_i < 0$	Local maximum
Indefinite, mixed signs	Saddle point
PSD/NSD with some $\lambda = 0$	Inconclusive

This is the multivariate second-derivative test.

3 The Chain Rule: Why It Is Backpropagation

Scalar chain rule:

$$\frac{d}{dx} [f(g(x))] = f'(g(x))g'(x).$$

Vector version, using Jacobians:

$$\frac{d}{dx} [f(g(x))] = J_f(g(x))J_g(x),$$

a matrix product evaluated right-to-left.

A neural network is a deep composition

$$L = \ell \circ f_L \circ f_{L-1} \circ \cdots \circ f_1.$$

The chain rule gives

$$\frac{\partial L}{\partial x} = \frac{\partial \ell}{\partial a_L} \frac{\partial a_L}{\partial a_{L-1}} \cdots \frac{\partial a_1}{\partial x}.$$

Backpropagation is just this product, computed right-to-left and cached.

3.1 Mini-Walkthrough: One-Layer Neural Network

Forward:

$$z = Wx + b, \quad a = \sigma(z), \quad \ell = \frac{1}{2}(a - y)^2.$$

Backward for one example, applying the chain rule layer by layer:

$$\begin{aligned} \frac{\partial \ell}{\partial a} &= a - y, & \frac{\partial a}{\partial z} &= \sigma'(z) = \sigma(z)(1 - \sigma(z)), \\ \frac{\partial z}{\partial W} &= x^\top, & \frac{\partial z}{\partial b} &= 1, & \frac{\partial z}{\partial x} &= W^\top. \end{aligned}$$

Then

$$\begin{aligned} \frac{\partial \ell}{\partial z} &= \frac{\partial \ell}{\partial a} \frac{\partial a}{\partial z} = (a - y)\sigma'(z), \\ \frac{\partial \ell}{\partial W} &= \frac{\partial \ell}{\partial z} \frac{\partial z}{\partial W} = (a - y)\sigma'(z)x^\top, & \frac{\partial \ell}{\partial b} &= \frac{\partial \ell}{\partial z}, \\ & & \frac{\partial \ell}{\partial x} &= W^\top \frac{\partial \ell}{\partial z}. \end{aligned}$$

The forward pass computes and caches z, a, ℓ . **The backward pass** multiplies local Jacobians from the loss back toward the inputs, reusing the cached intermediates.

Scale this up to L layers: each layer's gradient is the downstream gradient from the layer above times its own local Jacobian. That product structure is the chain rule, layer by layer. **No magic.**

4 Convexity: Why It Is the Property Everyone Wants

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is **convex** if for every x, y and every $t \in [0, 1]$,

$$f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y).$$

Geometrically, the line segment between any two points on the graph lies on or above the graph. **Equivalent characterizations** for differentiable f :

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) \quad (\text{first-order: tangent under graph}),$$

$$\nabla^2 f(x) \succeq 0 \quad \text{for all } x \quad (\text{second-order: Hessian PSD}).$$

Strictly convex means the inequalities are strict when appropriate.

4.1 The Killer Consequence

If f is convex:

- Every **local minimum** is also a **global minimum**.
- If f is strictly convex, the minimum is **unique**.
- Gradient descent, with appropriate step size, **provably converges to the global optimum**.

This is the structural difference between **SVM training**, a convex quadratic program with one global optimum, and **NN training**, which is non-convex and depends on initialization and optimizer trajectory.

4.2 Useful Convex Functions to Recognize

$$\begin{aligned}x^\top Px & \text{ with } P \succeq 0, & \|x\|_p & \text{ any norm,} \\ \max(0, x) & \text{ hinge/ReLU,} & \log(1 + e^x) & \text{ softplus,} \\ -\log x & (x > 0), & e^x & .\end{aligned}$$

Operations preserving convexity: nonnegative weighted sums, pointwise maximum, and composition with affine maps. So:

- Hinge loss $\max(0, 1 - y(w^\top x + b))$ is convex in (w, b) .
- Logistic loss $\log(1 + \exp(-y(w^\top x + b)))$ is convex.
- L_1 and L_2 regularizers added to a convex loss remain convex.
- A sum of convex losses is convex, so ERM with these losses is convex optimization.

4.3 Jensen's Inequality

For convex f and any random variable X ,

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)].$$

For concave f , reverse the inequality. This is used everywhere, including in the $\text{KL} \geq 0$ proof in the information theory notes.

5 Gradient Descent: The Algorithm

The update rule behind SL3 and all of OL2:

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k).$$

$\eta > 0$ is the **learning rate** or step size.

5.1 The Learning Rate Is the Parameter

η too large	overshoot, oscillate, diverge
η “just right”	smooth descent
η too small	crawl, stall on flat regions

There is a precise version: for an L -Lipschitz-smooth convex objective, i.e. $\|\nabla^2 f\| \leq L$, gradient descent converges iff $\eta < 2/L$, and converges fastest at $\eta = 1/L$. In practice you tune η , or schedule it, because L is unknown.

5.2 What Can Go Wrong on Non-Convex Surfaces

- **Local minima:** $\nabla f = 0$ but not the global best. In high-dimensional NN loss surfaces, true local minima are rare; most critical points are saddles.
- **Saddle points:** $\nabla f = 0$ but Hessian is indefinite. Pure SGD escapes these slowly because the gradient is small near a saddle.
- **Plateaus:** long flat regions; gradient is tiny, progress is glacial.
- **Ravines:** steep along one axis, flat along another; vanilla SGD oscillates without making progress along the flat direction.

OL2's tricks address all of these.

5.3 SGD, Momentum, RMSProp, Adam, AdamW: The Lineage

SGD with a single example or mini-batch:

$$\theta \leftarrow \theta - \eta \nabla L_{\text{batch}}(\theta).$$

Noisier gradient than full-batch GD but cheaper per step, and the noise sometimes helps escape saddles. SGD's noise is what makes "implicit regularization" claims work in deep learning.

Momentum: accumulate an exponentially weighted gradient:

$$v \leftarrow \beta v + \nabla L(\theta), \quad \theta \leftarrow \theta - \eta v.$$

Averaging direction over many steps cuts ravine oscillations and builds inertia through plateaus. $\beta \approx 0.9$ is standard.

RMSProp: per-parameter adaptive step size based on a running average of squared gradients:

$$s \leftarrow \beta s + (1 - \beta)(\nabla L)^2, \quad \theta \leftarrow \theta - \eta \frac{\nabla L}{\sqrt{s} + \varepsilon}.$$

This damps updates along high-curvature axes and accelerates along flat ones. It solves the ravine problem from the adaptive-step angle, versus momentum's direction-averaging angle.

Adam: momentum + RMSProp + bias correction:

$$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla L, \quad v \leftarrow \beta_2 v + (1 - \beta_2) (\nabla L)^2, \\ \hat{m} = \frac{m}{1 - \beta_1^t}, \quad \hat{v} = \frac{v}{1 - \beta_2^t}, \quad \theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}.$$

Typical defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

AdamW: decouples weight decay from the gradient step, L_2 -as-prior regularization on parameters:

$$\theta \leftarrow \theta - \eta \left(\frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon} + \lambda \theta \right).$$

This is the patch that fixed Adam's poor generalization compared to SGD+momentum on many vision benchmarks. **AdamW** is "Adam where the L_2 actually behaves like L_2 ."

6 Constrained Optimization and Lagrange Multipliers

Many ML problems minimize an objective **subject to constraints**:

$$\begin{aligned} & \text{minimize} && f(x) \\ & \text{subject to} && g_i(x) \leq 0, \quad i = 1, \dots, m, \\ & && h_j(x) = 0, \quad j = 1, \dots, p. \end{aligned}$$

The **Lagrangian** folds constraints into the objective with multipliers:

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \nu_j h_j(x), \quad \lambda_i \geq 0.$$

λ_i is the multiplier, or shadow price, for the i -th inequality constraint; ν_j is for the j -th equality constraint.

6.1 KKT Conditions: What Optimality Looks Like with Constraints

For a convex problem with strong duality, e.g. satisfying Slater's condition, x^* is optimal iff there exist λ^*, ν^* such that:

$$\begin{aligned} \text{(KKT-1)} \quad & \nabla_x \mathcal{L}(x^*, \lambda^*, \nu^*) = 0 && \text{stationarity,} \\ \text{(KKT-2)} \quad & g_i(x^*) \leq 0, \quad h_j(x^*) = 0 && \text{primal feasibility,} \\ \text{(KKT-3)} \quad & \lambda_i^* \geq 0 && \text{dual feasibility,} \\ \text{(KKT-4)} \quad & \lambda_i^* g_i(x^*) = 0 && \text{complementary slackness.} \end{aligned}$$

The last condition, **complementary slackness**, is the load-bearing fact for SVM/support vectors: either $\lambda_i^* = 0$ (constraint inactive) or $g_i(x^*) = 0$ (constraint binding). So only binding constraints have nonzero multipliers.

6.2 The SVM as a Lagrangian: Full Derivation

Primal:

$$\begin{aligned} & \text{minimize} && \frac{1}{2} \|w\|^2 \\ & \text{subject to} && y_i(w \cdot x_i + b) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

Rewrite each constraint as $1 - y_i(w \cdot x_i + b) \leq 0$. Lagrangian:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i [y_i(w \cdot x_i + b) - 1], \quad \alpha_i \geq 0.$$

Stationarity: set gradients to zero:

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_i \alpha_i y_i x_i = 0 \quad \Rightarrow \quad w = \sum_i \alpha_i y_i x_i,$$

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_i \alpha_i y_i = 0 \quad \Rightarrow \quad \sum_i \alpha_i y_i = 0.$$

Substitute back into $\mathcal{L}$ to eliminate w and b :

$$\mathcal{L}_{\text{dual}}(\alpha) = \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i \cdot x_j).$$

The SVM dual:

$$\begin{aligned} & \text{maximize} && \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \\ & \text{subject to} && \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0. \end{aligned}$$

Three things to notice:

1. **Data appears only through inner products** $x_i \cdot x_j$. Replace these with a kernel $k(x_i, x_j)$ and you have the kernel trick. SL6.
2. **Complementary slackness** says $\alpha_i[y_i(w \cdot x_i + b) - 1] = 0$. So $\alpha_i > 0$ only when $y_i(w \cdot x_i + b) = 1$, meaning the point is on the margin. These are the **support vectors**. Everything else has $\alpha_i = 0$ and does not contribute to $w = \sum_i \alpha_i y_i x_i$: sparsity in the data, for free.
3. **Convexity of the dual** guarantees a unique optimum and reproducibility: exactly the SL6 vs. SL3 contrast.

6.3 Duality in One Sentence

Primal: minimize the original objective. Dual: maximize the Lagrangian after eliminating primal variables. Under convexity plus Slater, primal optimum equals dual optimum, **strong duality**, and the dual is often easier: lower-dimensional, or expressible only via inner products as in the SVM case.

7 Gradient-Based vs. Gradient-Free: The OL1 Connection

Everything above assumes you have a gradient. When you do not, because f is discontinuous, a black box, or combinatorial, you fall back to the **gradient-free** methods of OL1:

Method	What it explores	When to reach for it
Random restarts + hill climbing	Local moves, restart on local optimum	Cheap baseline
Simulated annealing	Local moves with random up-hill accepted, cooling	Rugged landscapes
Genetic algorithms	Population, crossover, mutation	Combinatorial / structured
MIMIC	Maintains a distribution over good solutions; updates dependency model	Structured problem with discoverable inter-dependencies

The calculus notes and OL1 are two halves of one picture: use the gradient when it exists and is informative; reach for randomized optimization when it does not.

8 Newton's Method: Preview

Gradient descent uses only first-order information. **Newton's method** uses the Hessian:

$$\theta_{k+1} = \theta_k - \left[\nabla^2 L(\theta_k) \right]^{-1} \nabla L(\theta_k).$$

It converges in one step on a quadratic and super-linearly near a local optimum. Cost: a Hessian inversion per step, $O(n^3)$, so it is impractical for large neural networks. **Quasi-Newton** methods such as L-BFGS approximate the Hessian and are widely used in classical ML, e.g. the default in scikit-learn's `LogisticRegression`.

9 Where It Shows Up in CS7641

Concept	Where
Chain rule	SL3 backpropagation
Gradient descent, learning rate	SL3; OL2 (SGD to momentum to Adam to AdamW)
Hessian, second-order test	OL2 (escape saddles, condition number)
Convex vs. non-convex	SL6 (SVM, convex) vs. SL3 (NN, non-convex)
$\nabla f = 0$ to find optima	MLE/MAP derivations (OL4, probability notes); OLS normal equations
Lagrange multipliers / KKT / duality	SL6 SVM dual, support vectors, kernel trick
Jensen's inequality	KL ≥ 0 (information theory); EM's lower bound (UL1)
When no gradient exists	OL1 randomized optimization

10 Review Questions with Sketched Answers

Conceptual Checks

1. Gradient geometry; why subtract it?

∇f points in the direction of steepest **ascent**, with magnitude equal to the rate of ascent. The proof is that the directional derivative $\nabla f \cdot u$ is maximized when $u \parallel \nabla f$, by Cauchy–Schwarz. To minimize, move opposite to it: $\theta \leftarrow \theta - \eta \nabla f$.

2. “Backprop is the chain rule” in one sentence.

Backpropagation applies the chain rule to the loss as a deep composition of layers, multiplying local Jacobians from output to input. The forward pass computes and caches activations; the backward pass multiplies downstream gradients by local Jacobians at each layer, reusing those caches.

3. Definition of convex; single most important consequence.

The chord between any two points on the graph lies on or above the graph, equivalently Hessian PSD for differentiable f . **Every local minimum is the global minimum**, so gradient descent finds the global optimum with no local-minima trap or initialization sensitivity.

Derivation / Math

4. Chain rule for $f(u) = u^2$, $g(x) = 3x + 1$; scale to NN.

$$\frac{d}{dx}(3x + 1)^2 = 2(3x + 1) \cdot 3 = 6(3x + 1).$$

By the same logic, in a multi-layer NN each layer's gradient equals the downstream gradient times that layer's local Jacobian; the full gradient is a chain of matrix products. Backprop organizes that chain right-to-left so intermediate results are reused.

5. **GD trajectory for η too large vs. too small.**

Too large: overshoots the minimum, oscillates around it, and diverges if the curvature is high enough. Too small: takes microscopic steps, stalls on plateaus, and never converges in reasonable time. Adaptive optimizers such as RMSProp and Adam automate the per-parameter trade-off.

6. **SVM as constrained optimization to Lagrangian to role of α_i .**

Primal: $\min \frac{1}{2} \|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$. Lagrangian:

$$\mathcal{L} = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i [y_i(w \cdot x_i + b) - 1], \quad \alpha_i \geq 0.$$

Setting $\partial \mathcal{L} / \partial w = 0$ gives $w = \sum_i \alpha_i y_i x_i$. Complementary slackness $\alpha_i [y_i(w \cdot x_i + b) - 1] = 0$ means $\alpha_i > 0$ only for points on the margin: the **support vectors**.

Analysis: Why Does It Behave This Way?

7. **SVM global optimum vs. NN local optima: name the property.**

Convexity. SVM training is a convex quadratic program with PSD Hessian, so one global optimum exists and is found regardless of initialization. NN training is non-convex, so different initializations land in different solutions; seeded reproducibility is the best you can do.

8. **Two things that determine NN solution; OL2 mitigation.**

Initialization determines which basin you start in. Optimizer trajectory, including batch order, learning rate schedule, momentum, and adaptive updates, determines how you move. Mitigation: momentum or Adam smooths through plateaus and ravines; ensemble multiple initializations and average to reduce variance.

9. **Gradient existence as the OL1/OL2 divide.**

OL2 methods need a usable gradient: f differentiable and informative. For example, training a NN with squared loss. OL1 does not: f can be discontinuous, combinatorial, or a black box. Example: finding the best feature subset, with 2^d combinations and no gradient over a discrete set.

Exam-Style

10. **$\nabla f = 0$ closed-form derivation: name one.**

OLS regression. Loss $L(\beta) = \|X\beta - y\|^2$.

$$\nabla L = 2X^T(X\beta - y) = 0 \quad \Rightarrow \quad X^T X\beta = X^T y \quad \Rightarrow \quad \hat{\beta} = (X^T X)^{-1} X^T y.$$

Equivalently, this is MLE for Gaussian-noise linear regression. Bernoulli MLE follows the same $\nabla \log L = 0$ pattern.

11. **SVM chain: primal to Lagrangian to dual to kernel trick.**

Primal constrained QP $\rightarrow$ Lagrangian folds constraints in with $\alpha_i \geq 0 \rightarrow$ stationarity eliminates $w, b \rightarrow$ dual depends on data only through $x_i \cdot x_j \rightarrow$ replace with kernel $k(x_i, x_j)$, Mercer, to operate implicitly in a higher-dimensional space without computing ϕ . Each step buys tractable optimization, shadow-price interpretation, sparsity in support vectors, and nonlinearity for free.

12. **Convexity / chain rule / GD to CS7641 lectures.**

Convexity maps to SL6 SVM: PSD Hessian of the QP guarantees a unique global optimum. **Chain rule** maps to SL3 NN backpropagation: multilayer composition makes the chain rule load-bearing. **Gradient descent** maps to OL2 and SL3 NN training: standard iterative optimization for differentiable losses; momentum, Adam, and AdamW are the practical descendants.